

Abstraction

1. Create an abstract class Shape

```
public abstract class Shape {  
  
}
```

2. The Shape class has two abstract methods: calculateArea() and calculatePerimeter.
Both the methods have a return type of void

```
// Abstract class Shape  
abstract class Shape {  
    // Abstract methods  
    public abstract void calculateArea();  
    public abstract void calculatePerimeter();  
}
```

3. Create
a class Quadrilateral which extends the abstract class Shape.

```
// Quadrilateral class extending Shape  
class Quadrilateral extends Shape {  
    // Implementing abstract methods  
    @Override  
    public void calculateArea() {  
        System.out.println("Calculating area of Quadrilateral");  
    }  
  
    @Override  
    public void calculatePerimeter() {  
        System.out.println("Calculating perimeter of Quadrilateral");  
    }  
}
```

4. Implement all the abstract method of the parent class

```
// Abstract class Shape

abstract class Shape {

    // Abstract methods

    public abstract void calculateArea();

    public abstract void calculatePerimeter();

}

// Quadrilateral class extending Shape

class Quadrilateral extends Shape {

    // Implementing abstract methods

    @Override

    public void calculateArea() {

        // Implementation for calculating area of a quadrilateral

        System.out.println("Calculating area of Quadrilateral");

        // Actual calculation would depend on specific type of
        quadrilateral

    }

    @Override

    public void calculatePerimeter() {

        // Implementation for calculating perimeter of a quadrilateral
```

```
        System.out.println("Calculating perimeter of Quadrilateral");

        // Actual calculation would depend on specific type of
        quadrilateral

    }

}

// Main class to demonstrate

public class AbstractMethod {

    public static void main(String[] args) {

        Quadrilateral quad = new Quadrilateral();

        quad.calculateArea();

        quad.calculatePerimeter();

    }

}
```

Assuming and Taking the values to implement Formula and Logics:

```
abstract class Shape {
    // Abstract methods
    public abstract void calculateArea();
    public abstract void calculatePerimeter();
}

class Quadrilateral extends Shape {
    private double length;
    private double width;
```

```
public Quadrilateral(double length, double width) {
    this.length = length;
    this.width = width;
}

@Override
public void calculateArea() {
    double area = length * width;
    System.out.println("Area of Quadrilateral (Rectangle): " + area);
}

@Override
public void calculatePerimeter() {
    double perimeter = 2 * (length + width);
    System.out.println("Perimeter of Quadrilateral (Rectangle): " +
perimeter);
}
}

// Main class to test
public class AbstractMethod {
    public static void main(String[] args) {
        Quadrilateral rectangle = new Quadrilateral(5.0, 3.0);
        rectangle.calculateArea();
        rectangle.calculatePerimeter();
    }
}
```

```
Cannot load PSReadline module. Console is running without PSReadline.
PS E:\Jaca\Dev\workshop-5> & "C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe" "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\Vajju Shrestha\AppData\Roaming\Code\User\workspaceStorage\lab7c4261ef0250206afad85c8ff6c2e86\redhat-.java\jdk_ws\1\
ava\workshop-5_8813de51\bin" "AbstractMethod"
Area of Quadrilateral (Rectangle): 15.0
Perimeter of Quadrilateral (Rectangle): 16.0
PS E:\Jaca\Dev\workshop-5>
```

5. Create an abstract class named Vehicle which consist of two methods: wheel and door. Both the methods have void return type and no parameters. The method wheel has no implementation.

```
// Abstract class Vehicle
abstract class Vehicle {
    // Abstract method wheel (no implementation)
    public abstract void wheel();

    // Concrete method door (has implementation)
    public void door() {
        System.out.println("This vehicle has doors.");
    }
}
```

6. Create a class name Bus and extend the Vehicle class.

```
// Bus class extending Vehicle

class Bus extends Vehicle {

    @Override

    public void wheel() {

        System.out.println("A bus has 6 wheels.");

    }

    @Override

    public void door() {

        System.out.println("A bus has 2 doors.");

    }

}
```

Full Code will Be :

```
abstract class Vehicle {
    public abstract void wheel();
    public void door() {
        System.out.println("This vehicle has doors.");
    }
}

class Bus extends Vehicle {
    @Override
    public void wheel() {
        System.out.println("A bus has 6 wheels.");
    }
}
```

```
    }

    @Override
    public void door() {
        System.out.println("A bus has 2 doors.");
    }
}

public class Main {
    public static void main(String[] args) {
        Bus myBus = new Bus();
        myBus.wheel();
        myBus.door();
    }
}
```

```
PS E:\Jaca\Javaworkshop-5>
PS E:\Jaca\Javaworkshop-5> cd 'e:\Jaca\Javaworkshop-5'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Vaijla Shrestha\AppData\Roaming\Code\User\workspaceStorage\lab7c\4261ef0250206a6a95cbf6c2e86\redhat.java\jdt_ws\Javaworkshop-5.8813de51\bin' 'Main'
A bus has 6 wheels.
A bus has 2 doors.
PS E:\Jaca\Javaworkshop-5> █
```

Interface

7. Create an interface Animal. The Animal interface has two methods eat() and walk()

```
// Animal interface
public interface Animal {
    void eat(); // Abstract method (no body)
    void walk(); // Abstract method (no body)
}
```

8. Create another interface Printable. The Printable interface has a method called display();

```
// Printable interface
```

```
public interface Printable {  
    void display(); // Abstract method (no body)  
}
```

9. Create a class Cow that implements the Animal and Printable interfaces

```
// Animal interface  
interface Animal {  
    void eat();  
    void walk();  
}  
  
// Printable interface  
interface Printable {  
    void display();  
}
```

Full Code :

```
// Animal interface  
interface Animal {  
    void eat();  
    void walk();  
}  
  
// Printable interface  
interface Printable {  
    void display();  
}  
  
// Cow class implementing Animal and Printable  
class Cow implements Animal, Printable {  
    @Override  
    public void eat() {  
        System.out.println("Cow eats grass.");  
    }  
  
    @Override  
    public void walk() {
```



```
        System.out.println("Cow walks slowly on four legs.");
    }

    @Override
    public void display() {
        System.out.println("Displaying cow details: Herbivore, Domestic
Animal.");
    }
}

// Main class to test
public class Main {
    public static void main(String[] args) {
        Cow myCow = new Cow();

        // Animal interface methods
        myCow.eat();
        myCow.walk();

        // Printable interface method
        myCow.display();
    }
}
```

```
Cannot load PSReadline module. Console is running without PSReadline.
PS E:\Jaca\Javaworkshop-5> & "C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe" "-Xmx<ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\Vaifu Shrestha\AppData\Roaming\Code\User\workspaceStorage\ab7c4261ef0250208a6a05cbfc2e80\redhat_java\jdk_vs\
javaworkshop-5_5813de51\bin" "Main"
Cow eats grass.
Cow walks slowly on four legs.
Displaying cow details: Herbivore, Domestic Animal.
PS E:\Jaca\Javaworkshop-5>
```

10. Create an interface LivingBeing

```
public interface LivingBeing {
    void breathe();
    void grow();
}
```

11. Create an method void specialFeature()

```
public void specialFeature() {  
    System.out.println("Special Feature: Cow has four stomachs for  
digestion!");  
}
```

```
// LivingBeing interface  
interface LivingBeing {  
    void breathe();  
    void grow();  
}  
  
// Animal interface extending LivingBeing  
interface Animal extends LivingBeing {  
    void eat();  
    void walk();  
}  
  
// Printable interface  
interface Printable {  
    void display();  
}  
  
// Cow class implementing both Animal and Printable  
class Cow implements Animal, Printable {  
    // LivingBeing methods  
    @Override  
    public void breathe() {  
        System.out.println("Cow breathes through lungs");  
    }  
  
    @Override  
    public void grow() {  
        System.out.println("Cow grows from calf to adult in 2 years");  
    }  
  
    // Animal methods  
    @Override  
    public void eat() {  
        System.out.println("Cow eats grass and hay");  
    }  
}
```

```
}

@Override
public void walk() {
    System.out.println("Cow walks slowly on four legs");
}

// Printable method
@Override
public void display() {
    System.out.println("Cow Information: Domestic animal, provides
milk");
}

// Special feature method
public void specialFeature() {
    System.out.println("Special Feature: Cow has four stomachs for
digestion!");
}
}

// Main class
public class Main {
    public static void main(String[] args) {
        Cow myCow = new Cow();

        // LivingBeing methods
        myCow.breathe();
        myCow.grow();

        // Animal methods
        myCow.eat();
        myCow.walk();

        // Printable method
        myCow.display();

        // Special feature
        myCow.specialFeature();
    }
}
```

```
}
```

```
Cannot load PSReadline module. Console is running without PSReadline.  
PS E:\Java\JavaWorkshop> A: C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe -XX:+ShowCodeDetailsInExceptionMessages -cp "C:\Users\Vaifu\Shrestha\AppData\Roaming\Code\User\workspaceStorage\ab7c4261ef0250206a6a05c8f6c2e80\redhat_java\jdk_x64\workspace-5.8813651\bin" Main  
Cow breathes through lungs  
Cow grows from calf to adult in 2 years  
Cow eats grass and hay  
Cow walks slowly on four legs  
Cow Information: Domestic animal, provides milk  
Special Feature: Cow has four stomachs for digestion!  
PS E:\Java\JavaWorkshop>
```

Classes

12. Create 2 classes Fish and Bird that implements LivingBeing

```
class Fish implements LivingBeing {  
    @Override  
    public void breathe() {  
        System.out.println("Fish breathes through gills");  
    }  
  
    @Override  
    public void grow() {  
        System.out.println("Fish grows from fry to adult");  
    }  
  
    public void specialFeature() {  
        System.out.println("Special Feature: Can breathe underwater using  
gills");  
    }  
}
```

13. The specialFeature should display special features of the respective class animal.

```
class Bird implements LivingBeing {

    @Override

    public void breathe() {

        System.out.println("Bird breathes through lungs");

    }

    @Override

    public void grow() {

        System.out.println("Bird grows from chick to adult");

    }

    public void specialFeature() {

        System.out.println("Special Feature: Can fly with wings");

    }

}
```

Full Code will be :

```
// LivingBeing interface
interface LivingBeing {
    void breathe();
    void grow();
}
```

```
// Fish class implementing LivingBeing
class Fish implements LivingBeing {
    @Override
    public void breathe() {
        System.out.println("Fish breathes through gills");
    }

    @Override
    public void grow() {
        System.out.println("Fish grows from fry to adult");
    }

    public void specialFeature() {
        System.out.println("Special Feature: Can breathe underwater using gills");
    }
}

// Bird class implementing LivingBeing
class Bird implements LivingBeing {
    @Override
    public void breathe() {
        System.out.println("Bird breathes through lungs");
    }

    @Override
    public void grow() {
        System.out.println("Bird grows from chick to adult");
    }

    public void specialFeature() {
        System.out.println("Special Feature: Can fly with wings");
    }
}

// Main class
public class MainBird {
    public static void main(String[] args) {
        // Fish demonstration
        Fish myFish = new Fish();
    }
}
```

```
        System.out.println("--- Fish Characteristics ---");
        myFish.breathe();
        myFish.grow();
        myFish.specialFeature();

        // Bird demonstration
        Bird myBird = new Bird();
        System.out.println("\n--- Bird Characteristics ---");
        myBird.breathe();
        myBird.grow();
        myBird.specialFeature();
    }
}
```

```
Cannot load PSReadline module. Console is running without PSReadline.
PS E:\Java\JavaWorkshop> & "C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe" "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\Vaifu\Shrestha\AppData\Roaming\Code\User\workspaceStorage\ab74261ef0250286a6ad05cbfc2e80\redhat-java\jdk_ws\JavaWorkshop-5_BallBetti\bin" "MainBird"
--- Fish Characteristics ---
Fish breathes through gills
Fish grows from fry to adult
Special Feature: Can breathe underwater using gills

--- Bird Characteristics ---
Bird breathes through lungs
Bird grows from chick to adult
Special Feature: Can fly with wings
PS E:\Java\JavaWorkshop> █
```

Exception

14. In the following program, which exception will be generated

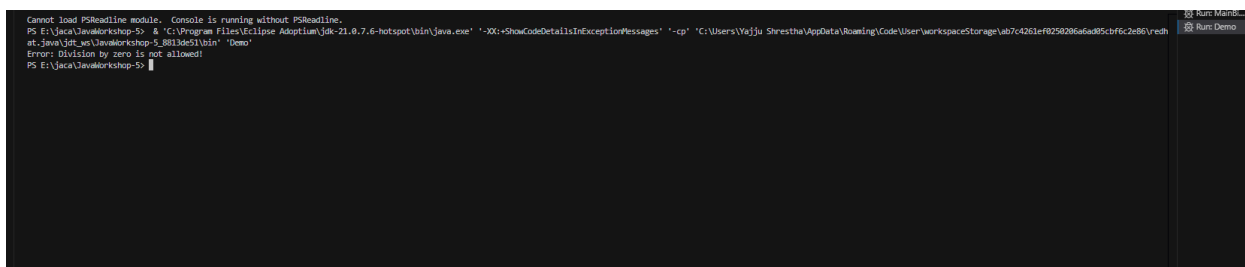
```
public class Demo{  
  
    public static void main(String[] args) {  
  
        System.out.println(10/0);  
  
    }  
  
}
```

Handle the exception above by using try-catch.

In the given program, the exception that will be generated is:

ArithmeticException - Specifically, it will be **ArithmeticException: / by zero** because division by zero is mathematically undefined and not allowed in Java.

```
public class Demo {  
    public static void main(String[] args) {  
        try {  
            System.out.println(10 / 0);  
        } catch (ArithmeticException e) {  
            System.out.println("Error: Division by zero is not allowed!");  
            // You can also print the exception details:  
            // e.printStackTrace();  
        }  
    }  
}
```



15. In the following program, which exception will be generated

```
public class Demo{  
    public static void main(String[] args) {  
        int[] age = {10,20,25,24,28,27,30,31,32};  
        System.out.println(age[9]);  
    }  
}
```

Handle the exception by using throws keyword.

In the given program, the exception that will be generated is:

ArrayIndexOutOfBoundsException - This occurs because the array `age` has indices from 0 to 8 (total 9 elements), but the code tries to access `age[9]` (the 10th element), which doesn't exist.

```
public class Demo {  
  
    public static void main(String[] args) throws  
ArrayIndexOutOfBoundsException {  
  
        int[] age = {10, 20, 25, 24, 28, 27, 30, 31, 32};  
  
        System.out.println(age[9]); // Exception will be thrown to the  
JVM  
  
    }  
  
}
```